Homework 1

Exercise 1

(1) $0^*(0+1)(0+1)^*$

Matching strings:
a, b, c, d

Motivation:
$(0+1)^*$ accepts everything, as long as there is one symbol caught by the $(0+1)$. Therefore, every string except ε is accepted.

(2) $(0+1)^*1(0+1)^*0(0+1)^*$

Matching strings:
b, c

Motivation:
The regular expression accepts any string where there is some 1 followed by some 0, consecutive or not or not.

(3) $(0+(0+1)^*)(0^*+1^*)$

Matching strings:
a, b, c, d, e

Motivation:
$(0+1)^*$ subsumes both 0 and (0^*+1^*), therefore we can simplify the regular expression to $(0+1)^*$, which trivially accepts all strings.

(4) $(0^*+1)(0+1^*)$

Matching strings:
a, d, e

Motivation:
The language of the regular expression can be thought of as the following: once we se 01, then we cannot see another 0. Conversely, once we see 10, we cannot see another 1.

Therefore, b doesn't match as we start with 10 but have another 1 as the last symbol. Likewise, c is also invalid because we start with 01, then immediately read a 0. e, the empty string, *is* accepted because we read neither 01 nor 10.

(5) $(0^* + 1^*)^*$

Matching strings:
a, b, c, d, e

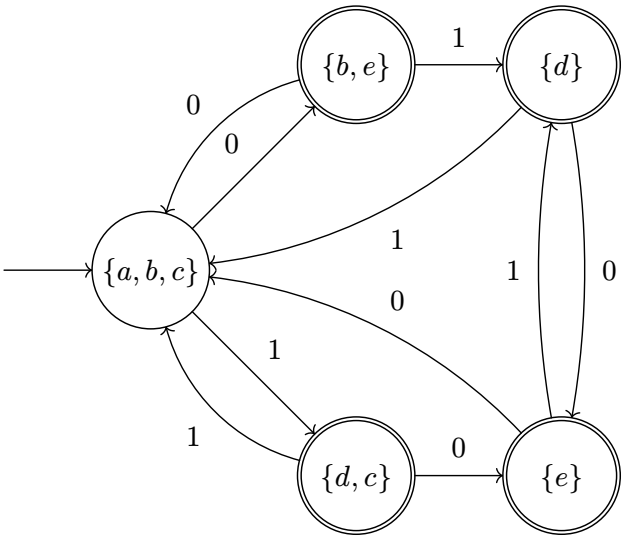
Motivation:
The regular expression trivially simplifies to $(0 + 1)^*$, which accepts all strings.

Exercise 2

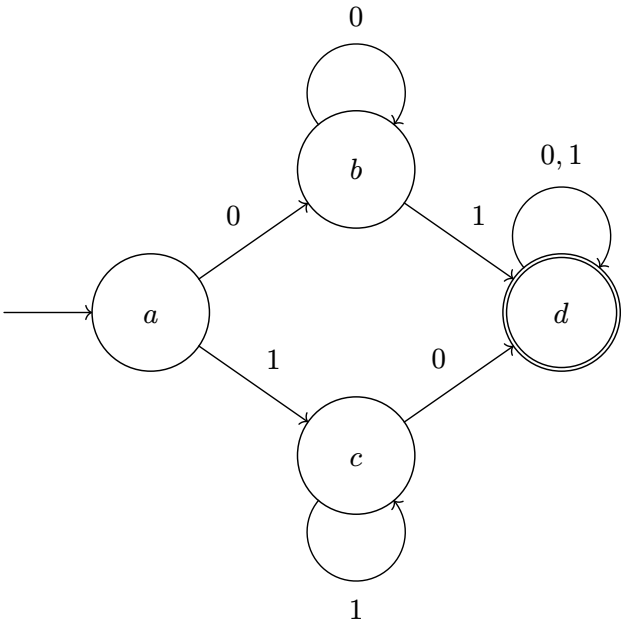
Step 1: present the result as a table. A short motivation is enough.
The below table was generated via the subset construction method over our DFA:

	0	**1**
$\{a, b, c\}$	$\{b, e\}$	$\{d, c\}$
$\{b, e\}$	$\{a, b, c\}$	$\{d\}$
$\{d, c\}$	$\{e\}$	$\{a, b, c\}$
$\{d\}$	$\{e\}$	$\{a, b, c\}$
$\{e\}$	$\{a, b, c\}$	$\{d\}$

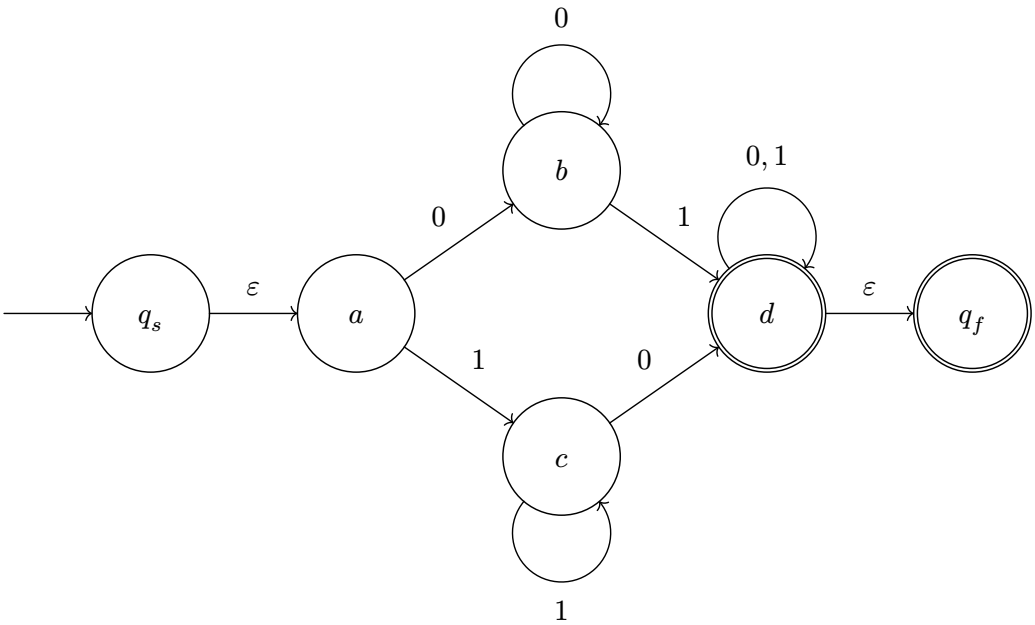
Step 2: draw the resulting DFA.



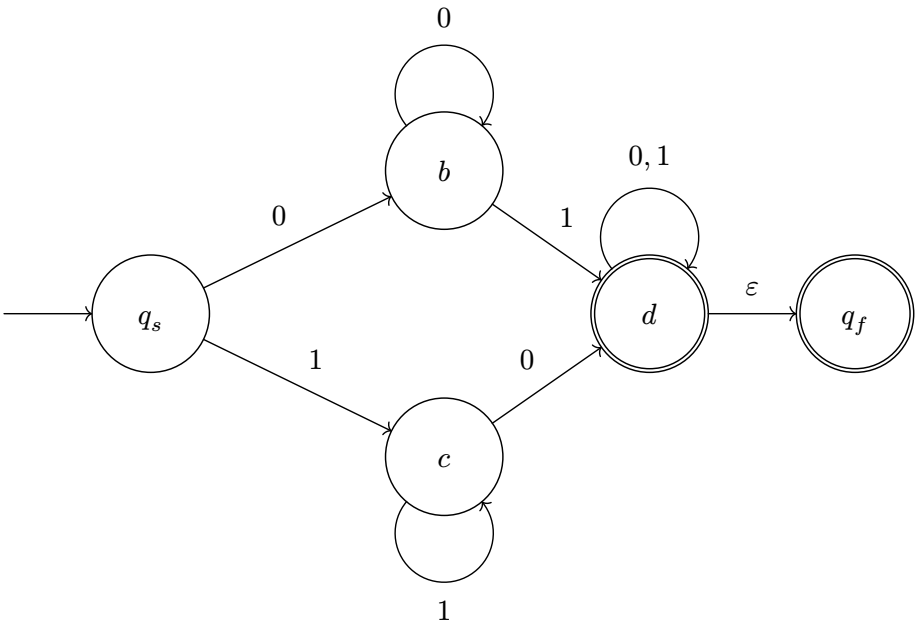
Exercise 3



Step 1: Add a new start and accept state.

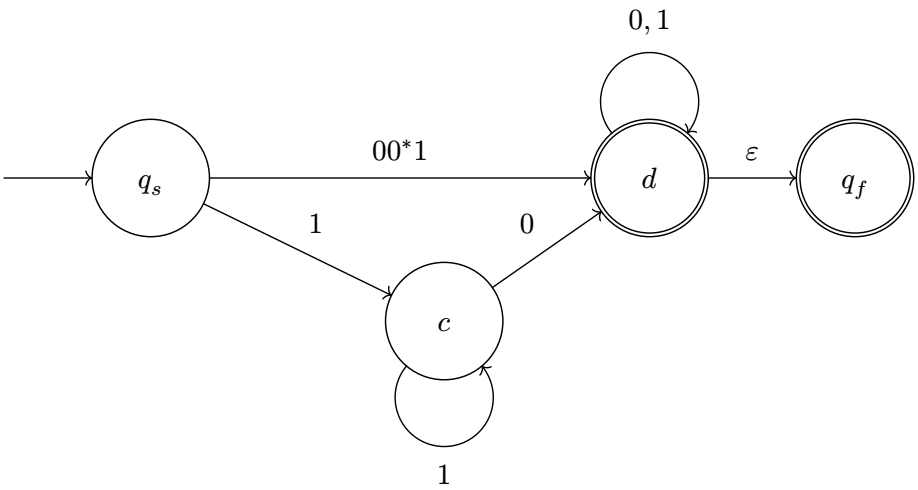


Step 2: Remove state a.



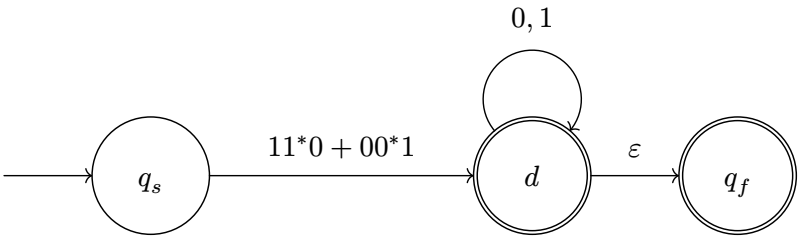
s_1	s_e	s_2	R_1	R_2	R_3	R_4	$R_1R_2^*R_3 + R_4$(simplified)
q_s	a	b	ε	$\emptyset$	0	$\emptyset$	0
q_s	a	c	ε	$\emptyset$	1	$\emptyset$	1

Step 3: Remove state b.



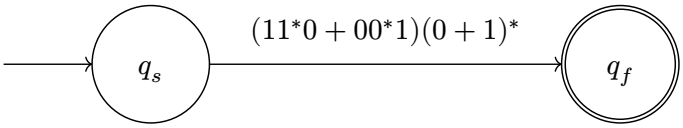
s_1	s_e	s_2	R_1	R_2	R_3	R_4	$R_1R_2^*R_3 + R_4$(simplified)
q_s	b	d	0	0	1	$\emptyset$	00*1

Step 4: Remove state c.



s_1	s_e	s_2	R_1	R_2	R_3	R_4	$R_1R_2^*R_3 + R_4$(simplified)
q_s	c	d	1	1	0	00^*1	$11^*0 + 00^*1$

Step 5 Remove state d.



s_1	s_e	s_2	R_1	R_2	R_3	R_4	$R_1R_2^*R_3 + R_4$(simplified)
q_s	d	q_f	$11^*0 + 00^*1$	$0 + 1$	ε	$\emptyset$	$(11^*0 + 00^*1)(0 + 1)^*$

Your regular expression:
$(11^*0 + 00^*1)(0 + 1)^*$

Exercise 4

Table after iteration 0:

a				
✓	b			
✓		c		
✓			d	
✓				e

Here we marked states which are not both accept or both not accept. This marks all pairs with $\{a\}$, as that is the only non-accept state.

Table/motivation after iteration 1

a				
✓	b			
✓	✓	c		
✓	✓		d	
✓		✓	✓	e

	0	1	Marked?
$\{b, c\}$	$\{e, a\}$	---	✓
$\{b, d\}$	$\{a, e\}$	---	✓
$\{b, e\}$	$\{a\}$	$\{d\}$	
$\{c, d\}$	$\{e\}$	$\{a\}$	
$\{c, e\}$	$\{e, a\}$	---	✓
$\{d, e\}$	$\{e, a\}$	---	✓

Table/motivation after iteration 2

a				
✓	b			
✓	✓	c		
✓	✓		d	
✓		✓	✓	e

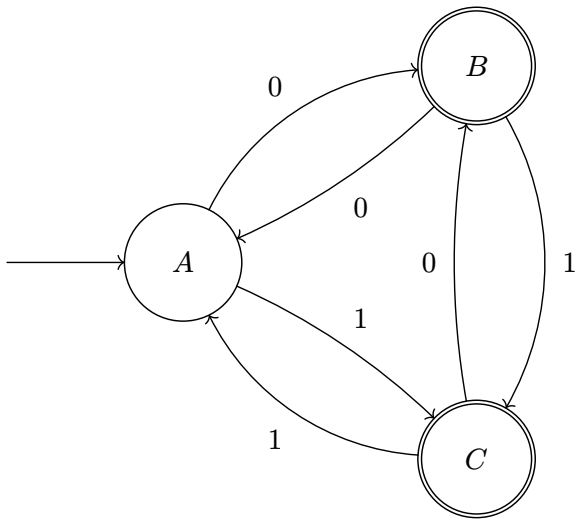
(the same as after iteration 1)

	0	1	Marked?
$\{b, e\}$	$\{a\}$	$\{d\}$	
$\{c, d\}$	$\{e\}$	$\{a\}$	

We learned no new information, so we terminate the algorithm and conclude that $b \equiv e$ and $c \equiv d$ under our DFA.

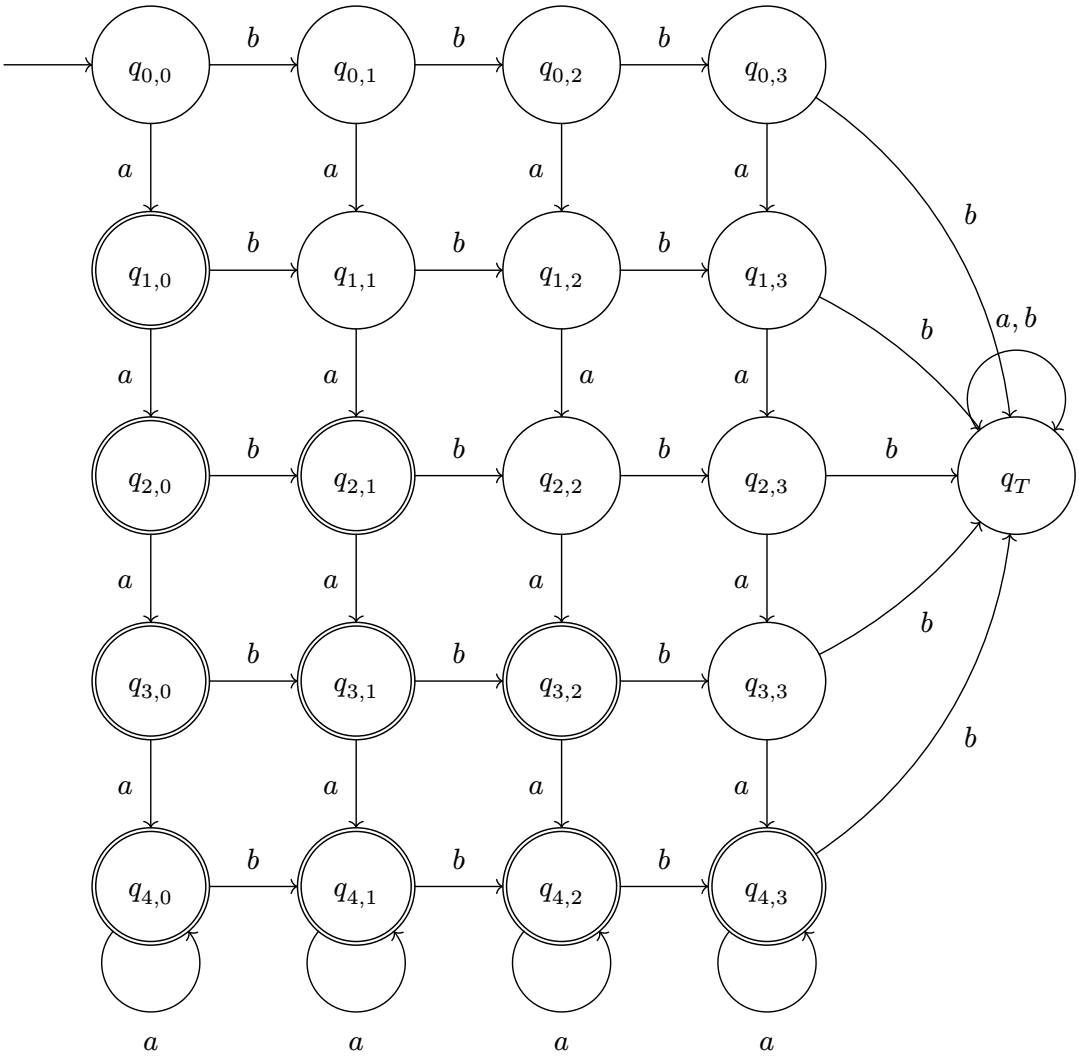
Minimized DFA (figure or transition table):
We rename/merge the states as follows:

- $a \to A$
- $b \to B$
- $c \to C$
- $d \to C$
- $e \to B$



Exercise 5

(a) DFA for L_1:



Motivation/explanation:
We have the states $q_{a,b}$, where $a \leq 4$ and $b \leq 3$, which we transition to after reading a or more a's, and b b's. The accept states are then naturally $\{q_{a,b} \mid b \leq 3 \wedge a > b\}$. We also have a trash state q_T for when we read more than 3 b's, as beyond that point the string is always rejected. We don't need to track the number of a's further than 4 since the $a > b$ condition will then always be satisfied, unless we reach 4 b's at which point we'll end up in q_T anyway.

(b)

Determine a suitable pumping length p:
We determine $p = 8$ from the longest (non-repeating) path in our DFA (for example reading $aaaabbbb$: $q_{0,0} \to q_{1,0} \to q_{2,0} \to q_{3,0} \to q_{4,0} \to q_{4,1} \to q_{4,2} \to q_{4,3} \to q_T$).

Choose a string $s \in L_1$ with $|s| \geq p$:
We choose the string $s = aaaaabbb$. s contains 5 a's and only 3 b's, so it's clear that $s \in L_1$.

Split $s = xyz$ such that $|xy| \leq p, |y| > 0$:
We split the string as following:

- $x = aaaa$
- $y = a$
- $z = bbb$

Provide an example of $i \neq 1$ such that $xy^iz \in L_1$:
We choose $i = 3$, yielding the string $xy^iz = aaaaaaabbb$. We only increased the number of a's in s, therefore we are still clearly in the language.

(c)

1. State your assumptions on the pumping length $p \geq 0$:
We need to prove the statement generally, therefore we cannot make any assumptions on the pumping length. We simply assume that L_2 has some arbitrary pumping length p.

2. Choose a string $s \in L_2$ such that $|s| \geq p$:
We choose $s = b^{p+1}a^{p+2}$. $\#a(s) > \#b(s)$ holds for any $p \geq 0$, therefore $s \in L_2$.

3. For every decomposition $s = xyz$ where $|xy| \leq p, |y| > 0$ find a value $i \geq 0$ such that $xy^iz \notin L_2$:
For all choices of x, y, z that satisfies the above conditions, the substring xy must only contain b's, since we defined s to start with $p + 1$ repeated b's and $|xy| \leq p$. Hence:

1. $y = b^m$ for some $m > 0$,
2. $x = b^k$ for some $k \geq 0$ such that $k + m \leq p$ and
3. $z = b^{p+1-k-m}a^{p+1}$.

This covers all possible decompositions $s = xyz$. To finish the proof, we need to show that there exists an $i \geq 0$ such that $xy^iz \notin L_2$. To generate a string that does not belong in our language, we need to pump enough b's to match the number of a's in s, that is $\#a(s) - \#b(xz)$. By definition $\#a(s) = p + 2$, and via (2, 3): $\#b(xz) = k + (p + 1 - k - m) = p + 1 - m$. For each increase in i, we generate m b's, therefore, to generate an invalid string $xy^iz = b^kb^{im}b^{p+1-k-m}a^{p+2}$, we need:

$$im \geq p + 2 - (p + 1 - m)$$

$$im \geq m + 1$$

$$i \geq \frac{m+1}{m}$$

We find $i = \left\lceil \frac{m+1}{m} \right\rceil$ generates a string xy^iz with the same number, or more, b's than a's, therefore $xy^iz \notin L_2$. Via the inverted pumping lemma, we conclude that L_2 is not regular.

(d) Use the Myhill-Nerode theorem from the lecture notes.
Consider the string a^ib^j where $i, j \in \mathbb{N}$. From the DFA of L_1, we see that $[a^ib^j] \underset{L_1}{\equiv} [a^{i+1}b^j]$ where $i \geq 4$, that is, reading one more a is equivalent with our current state. Conversely, $[a^ib^j] \underset{L_1}{\equiv} [a^ib^{j+1}]$ where $j \geq 3$. This means there a bound on the number of equivalence classes; once we've read 4 a's or 3 b's, we cannot reach any new classes. We conclude the number of equivalence classes is finite, which according to the Myhill-Nerode implies L_1 is regular.

The equivalence classes of L_2, on the other hand, lacks this constraint. Unlike L_1, reading a b beyond the third one is not going to lead us to an equivalent state, because in L_2 we then need to

read another a to reach an accept state. Formally, $[a^i b^j] \underset{L_2}{\not\equiv} [a^i b^{j+1}]$, which creates an infinite number of equivalence classes as j increases.